

A Modular Python Simulator for mmWave Vehicle-to-Infrastructure Link-Layer Research

mmWave V2I Simulator Contributors
Open-source research simulator

Abstract—We present an open-source Python simulator for mmWave vehicle-to-infrastructure (V2I) link-layer studies. The tool replaces a prior MATLAB workflow with a modular, reproducible architecture supporting synthetic and map-matched mobility, dual-band channel abstraction, runtime-selectable scheduling strategies, and an interactive desktop GUI inspired by the original vehicleBinPackSimulation.m visualization. The simulator targets researchers and practitioners who require license-free execution across macOS, Windows, and Linux.

Index Terms—mmWave, V2I, link-layer simulation, resource allocation, reproducible research

I. Introduction

Millimeter-wave V2I systems require joint evaluation of mobility, blockage, beam alignment, and time-frequency resource allocation. Prior MATLAB implementations demonstrated bin-packing-based allocation in urban vehicular scenarios but depend on licensed tooling and external executables.

This work provides a Python port with plugin interfaces for mobility, channel, scheduling, and visualization modules.

II. Architecture

The simulator is organized into deterministic core modules:

- Mobility: synthetic lane-grid and CSV map-matched routes with city presets.
- Channel: geometric LOS/NLOS and dual-band (28/39 GHz) path-loss abstraction.
- Scheduling: pluggable strategies (Max-CQI, proportional fair, latency-aware).
- GUI: MATLAB-style map, packing panel, and trim-loss trend charts.

III. Modeling Assumptions

Channel behavior uses simplified 3GPP-inspired path loss and geometric blockage. Scheduler/packing uses in-process Python algorithms (Guillotine, Shelf, MaxRects) rather than the original C++ rectangle-bin executable. Validation and density-sweep statistics are reproducible from fixed seeds with documented caveats.

IV. Interactive Visualization

The desktop application reproduces the original two-panel layout. Figure 1 shows the aerial map with vehicular routes, line-of-sight beams, and per-vehicle status coloring. Figure 2 shows the normalized resource-block packing view with trim-loss summary for the current timestep.

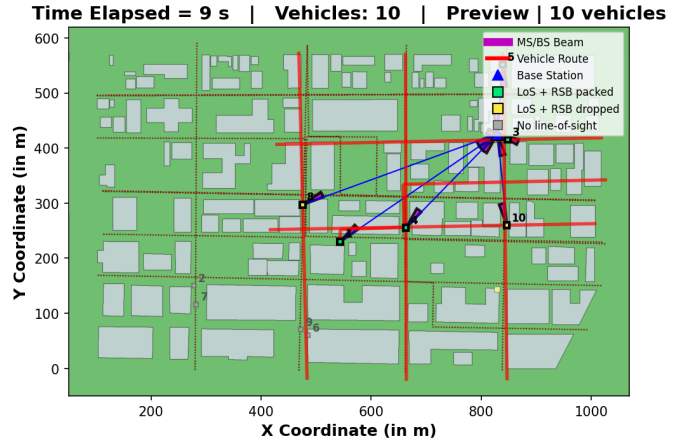


Fig. 1. Figure 1 style view: aerial map, active routes, LoS beams, and vehicle packing status.

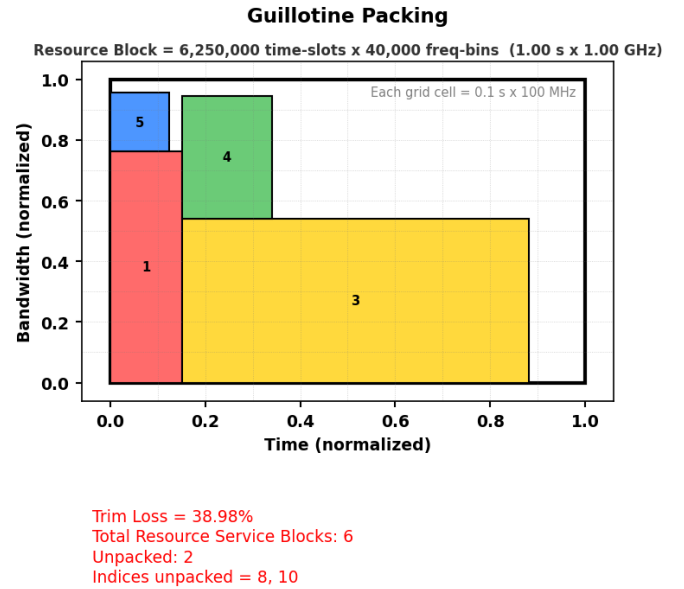


Fig. 2. Figure 2 style view: 2DRBP packing of resource service blocks and trim-loss readout.

V. Reproducibility

Artifacts are generated via python scripts/generate_readme_assets.py and bash scripts/build_paper.sh. Fixed seeds yield deterministic

simulation outputs and session export bundles.

VI. Conclusion

The Python simulator provides a research-grade, extensible foundation for mmWave V2I link-layer experimentation without MATLAB licensing constraints. The desktop release pairs MATLAB-style 2D visualization with ZIP session export; city presets and dual-band abstractions support comparative studies documented in the repository.

Acknowledgment

This Python port is inspired by the original mmWave V2I MATLAB simulator (MathWorks File Exchange 84948).

References

- [1] R. Subramanian, "A Resource Allocation Scheme for Multi-User MmWave Vehicle-to-Infrastructure Communication," Future Technologies Conference (FTC), 2018.
- [2] R. Subramanian, "Link Layer Simulator for mmWave V2I Network," MathWorks File Exchange, 2019.